2020년 겨울계절학기

# 네트워크프로그래밍 실습 & 과제 #5

20. 12. 29

#5. 멀티플렉싱 기반 서버, 입출력함수

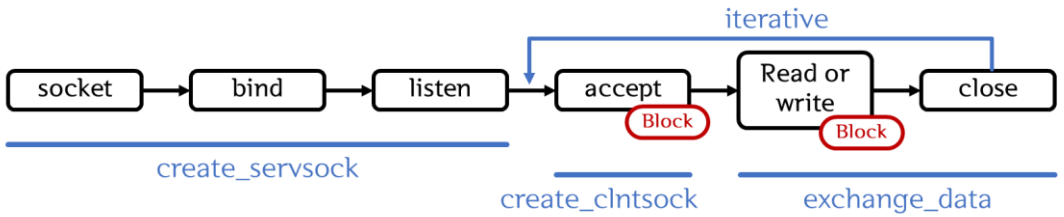
# 실습 #5

## 실습 내용

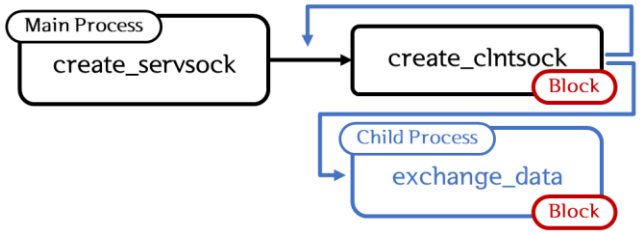
- Select()를 이용한 Concurrent Server 구현하기
- recv(), send() 함수 사용하기
- readv(), writev() 함수 사용하기

# 멀티플렉싱 프로그래밍에 필요한 작업

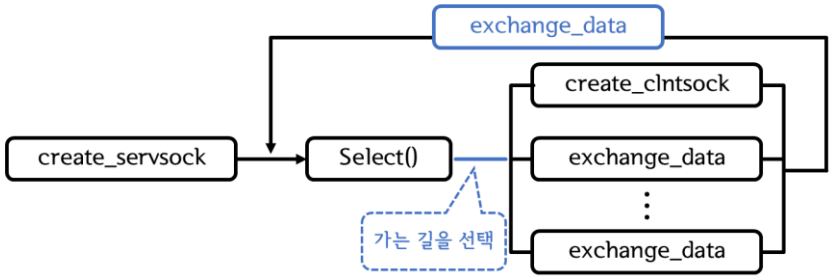
- Iterative




- Multi Process




- Multiplexing




- 관심 있는 파일 디스크립터 그룹 생성 및 변화사항 (이벤트) 감지
- 이벤트 종류 식별
- 이벤트 종류에 따른 적절한 처리 수행

# Select()를 이용한 Concurrent Server 구현하기

- 관심 있는 파일 디스크립터 그룹 생성 및 변화 사항 (event) 감지

**그룹 생성**

```
fd_set reads;          // fd_set은 128bit size를 가지기 때문에 파일 디스크립터 128개를 관리함
FD_ZERO(&reads);       // 초기화 코드. 추후 다른 파일 디스크립터를 등록할 때 호출하면 안됨
```

**특정 파일 디스크립터를 그룹에 넣기**

```
FD_SET(serv_sock, &reads);
fd_max = serv_sock;
```

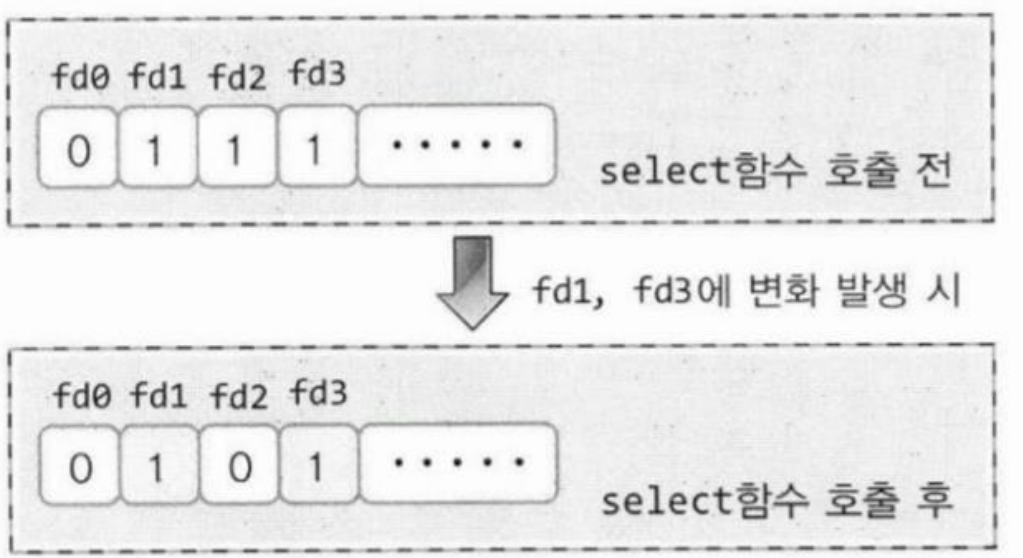
**Time-out 설정**

```
struct timeval timeout;

timeout.tv_sec = 5;          // seconds 단위
timeout.tv_usec = 500;       // microseconds 단위
```

```
if ((fd_num = select(fd_max + 1, &cpy_reads, 0, 0, &timeout)) == -1)
    break;
if (fd_num == 0){
    printf("timeout!!\n");
    continue;
}
```

# Select()를 이용한 Concurrent Server 구현하기

- 이벤트 종류 식별




- select 함수가 양수를 반환하면 그 수만큼 파일 디스크립터에 변화가 생겼음을 의미
- select 함수 호출이 완료되면 select 함수에 전달한 fd_set 변수 값에는 그림과 같은 변화가 발생
- 1로 설정된 모든 비트가 0으로 변경되지만, 변화가 발생한 파일 디스크립터에 해당하는 비트는 1이 그대로 남아있음

```
for (i = 0; i < fd_max + 1; i++)
{
    if (FD_ISSET(i, &cpy_reads))    // i 파일 디스크립터에 변화가 생기면
    {
        // 처리 루틴
    }
```

# Select()를 이용한 Concurrent Server 구현하기

- 이벤트 종류에 따른 적절한 처리 수행

```
if (i == serv_sock) // connection request!
{
    clnt_sock = create_clientsock(serv_sock);
    FD_SET(clnt_sock, &reads);
    if (fd_max < clnt_sock)
        fd_max = clnt_sock;
    printf("connected client: %d \n", clnt_sock);
}
else // read message!
{
    if (!exchange_data(i))
    {
        printf("disconnected client: %d \n", i);
        FD_CLR(i, &reads);
    }
}
```

serv_sock이 메시지를 받은 소켓이면 클라이언트 소켓을 생성

- clnt_sock이 메시지를 받은 소켓이면 데이터 전송 수행
- 만약 FIN 메시지를 수신하면 해당 파일 디스크립터를 관심 그룹에서 삭제

# echo_selectserv.c

- 실행 결과

```
➜                    ✗ ./echo_selectserv
9000
connected client: 7
connected client: 8
receive from 127.0.0.1 : Hello world
receive from 127.0.0.1 : Hello world
disconnected client: 8
disconnected client: 7
```

```
➜                    ✗ ./echo_selectclnt 1
27.0.0.1 9000
Connected...........
Input message(Q to quit): Hello world
Message from server: Hello world
Input message(Q to quit): q
➜                    ✗
```

```
➜                    ✗ ./echo_selectclnt
127.0.0.1 9000
Connected...........
Input message(Q to quit): Hello world
Message from server: Hello world
Input message(Q to quit): q
➜                    ✗
```

# recv(), send() 함수 사용하기

- 입출력 함수 정리

| 입력 함수 | 출력 함수 | 특징 |
|---|---|---|
| read() | write() | 주로 많이 사용되는 I/O 함수 |
| recvfrom() | sendto() | 주소가 연결되어 있지 않은 소켓으로 통신할 때 사용 |
| recv() | send() | 데이터 전송 시 플래그 또는 옵션을 추가해야 할 때 사용 |
| readv() | writev() | 한 번에 여러 배열이 있는 데이터를 전송할 때 사용 |

# recv(), send() 함수 사용하기

- 마지막 매개변수에 데이터 송수신 시 적용할 옵션 정보를 추가할 수 있음
- 옵션의 종류
  - MSG_OBB
  - MSG_PEEK
  - MSG_DONTROUTE
    - 패킷을 전송할 때 게이트웨이를 사용하지 않고 직접 연결된 네트워크를 통해서 호스트로 데이터를 전송함
    - 일반적인 경우에는 사용되지 않음, 네트워크를 진단하거나 라우팅 프로그램을 개발할 때 사용됨
  - MSG_DONTWAIT
  - MSG_WAITALL

# recv(), send() 함수 사용하기

## MSG_PEEK

```c
void exchange_data(int sock)
{
    char message[BUF_SIZE];
    int str_len;

    while (1)
    {
        fputs("Input message(Q to quit): ", stdout);
        fgets(message, BUF_SIZE, stdin);

        if (!strcmp(message, "q\n") || !strcmp(message, "Q\n"))
            break;

        str_len = strlen(message);
        send(sock, &str_len, sizeof(int), 0);
        send(sock, message, strlen(message), 0);

        recv(sock, &str_len, sizeof(int), 0);
        str_len = recv(sock, message, str_len, MSG_PEEK | MSG_DONTWAIT);
        message[str_len] = 0;
        printf("\nRead data with MSG_PEEK: %s", message);

        str_len = recv(sock, message, str_len, 0);
        message[str_len] = 0;
        printf("Message from server: %s\n", message);
    }
    close(sock);
}
```

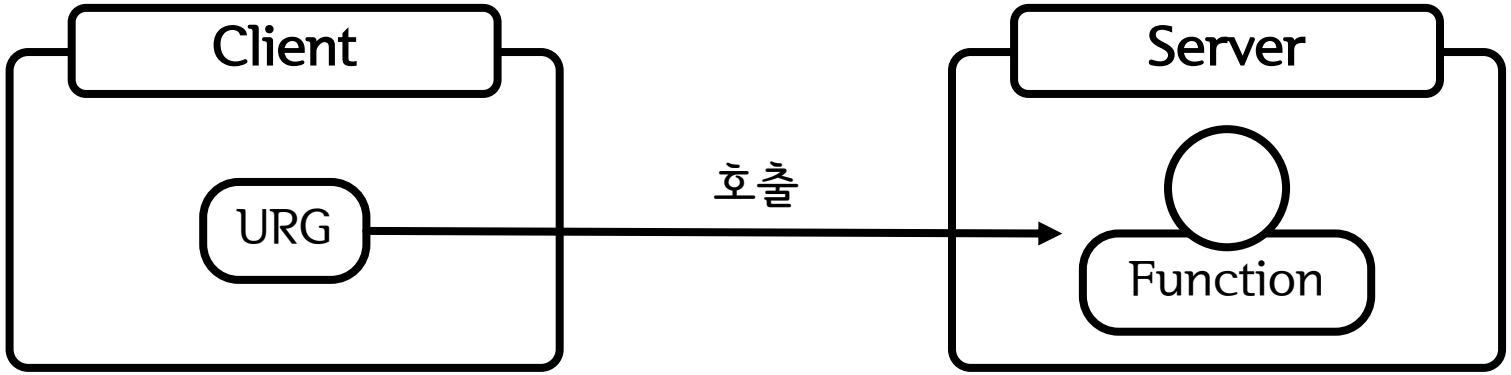
## 실행 결과

```
➜                    ✗ ./peek_serv 9000
connected client: 7
receive from 0.0.0.0 : Hello world
```

```
➜                    ✗ ./peek_clnt 127.0.0.1 9000
Connected...........
Input message(Q to quit): Hello world

Read data with MSG_PEEK: Hello world
Message from server: Hello world

Input message(Q to quit):
```

## recv(), send() 함수 사용하기

- MSG_OOB




- 상대방에 등록되어 있는 함수를 호출할 수 있음
- TCP 순서에 영향을 끼치지 않음! 단지 Callback을 호출하는 용도로 사용됨
- MSG_OOB 옵션과 함께 호출된 recv 함수는 1byte만 읽을 수 있음

# recv(), send() 함수 사용하기

- MSG_OOB (Server)

**이벤트 처리 함수 생성**

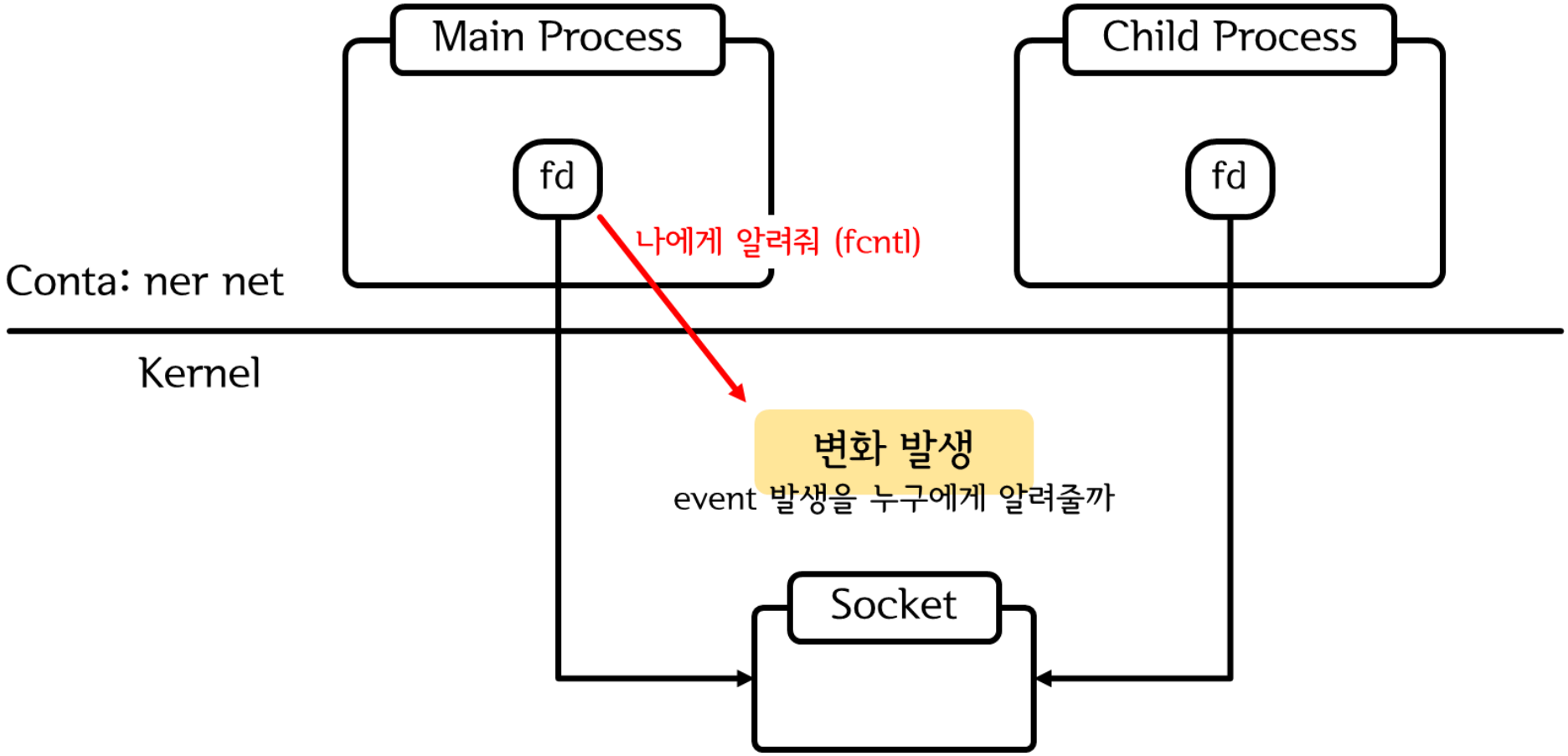
```c
// 처리 함수를 생성함
// Nodejs의 setTimeout 함수에 전달하는 function 객체와 같은 역할
void urg_handler(int signo)
{
    int str_len;
    char buf[BUF_SIZE];
    struct sockaddr_in clnt_adr;
    socklen_t clnt_adr_sz;

    str_len = recv(clnt_sock, buf, sizeof(buf) - 1, MSG_OOB);
    buf[str_len] = 0;
    printf("Urgent message: %s \n", buf);

    str_len = write(clnt_sock, "OK", strlen("OK"));
}
```

**이벤트 처리 함수 등록**

```c
// 처리 함수 등록
void urg_init()
{
    int state;
    struct sigaction act;
    act.sa_handler = urg_handler;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;

    fcntl(clnt_sock, F_SETOWN, getpid());
    state = sigaction(SIGURG, &act, 0);
}
```

# recv(), send() 함수 사용하기

- MSG_OOB (Server)



# recv(), send() 함수 사용하기

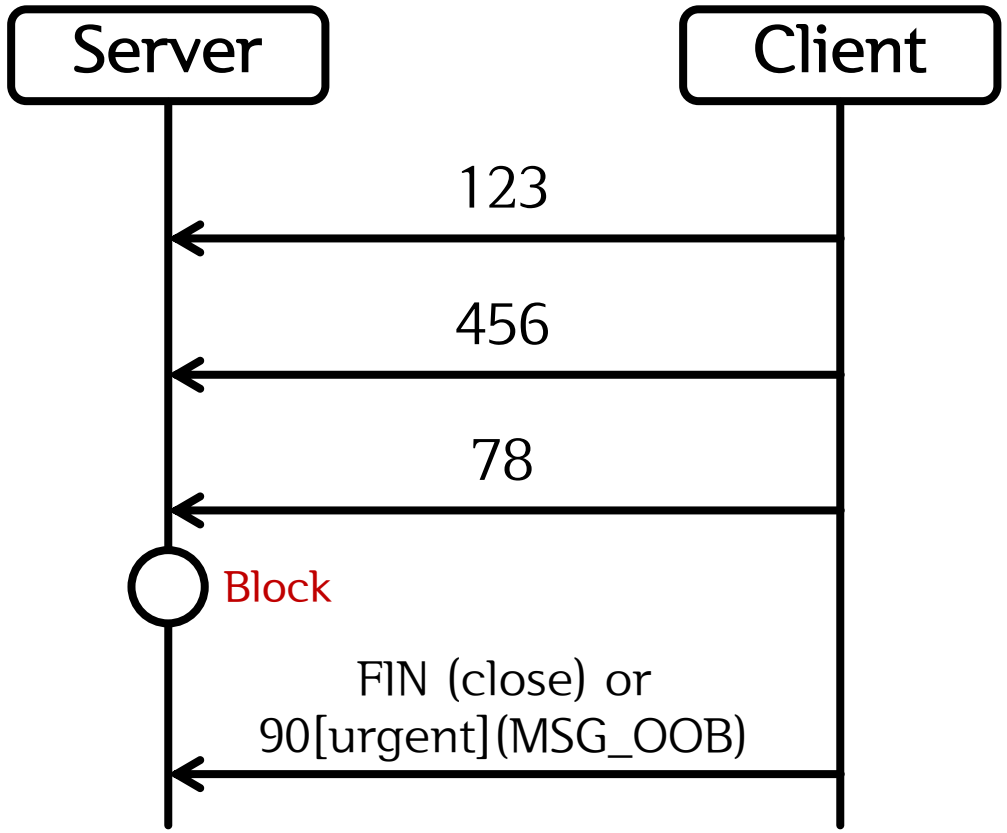
- MSG_OOB (Server)

```
void exchange_data()
{
    int str_len;
    char buf[BUF_SIZE];
    struct sockaddr_in clnt_adr;
    socklen_t clnt_adr_sz;

    while ((str_len = recv(clnt_sock, buf, sizeof(buf), MSG_WAITALL)) != 0)
    {
        if (str_len == -1)
            continue;
        buf[str_len] = 0;
        printf("exchange_data() : receive from %s : %s\n", \
inet_ntoa(clnt_adr.sin_addr), buf);
    }
}
```

# recv(), send() 함수 사용하기

- MSG_OOB (Client)

```c
void exchange_data(int sock)
{
    char message[BUF_SIZE];
    int str_len;

    write(sock, "123", strlen("123"));
    write(sock, "456", strlen("456"));
    write(sock, "78", strlen("78"));

    fputs("Input message(close or oob): ", stdout);
    fgets(message, BUF_SIZE, stdin);

    if (!strcmp(message, "close\n"))
        close(sock);
    else if (!strcmp(message, "oob\n"))
    {
        send(sock, "90", strlen("90"), MSG_OOB);
        str_len = read(sock, message, strlen("OK"));
        message[str_len] = 0;
        puts(message);
        sleep(5);
        puts("ByeBye!!");
        close(sock);
    }
}
```



# recv(), send() 함수 사용하기

- 실행 결과

close()

```
➜  ✗ ./oob_recv 9000
Connected client 1
exchange_data() : receive from 0.0.0.0 : 12345678
➜  ✗
```

```
➜  ✗ ./oob_send 127.0.0.1 9000
Connected...........
Input message(close or oob): close
➜  ✗
```

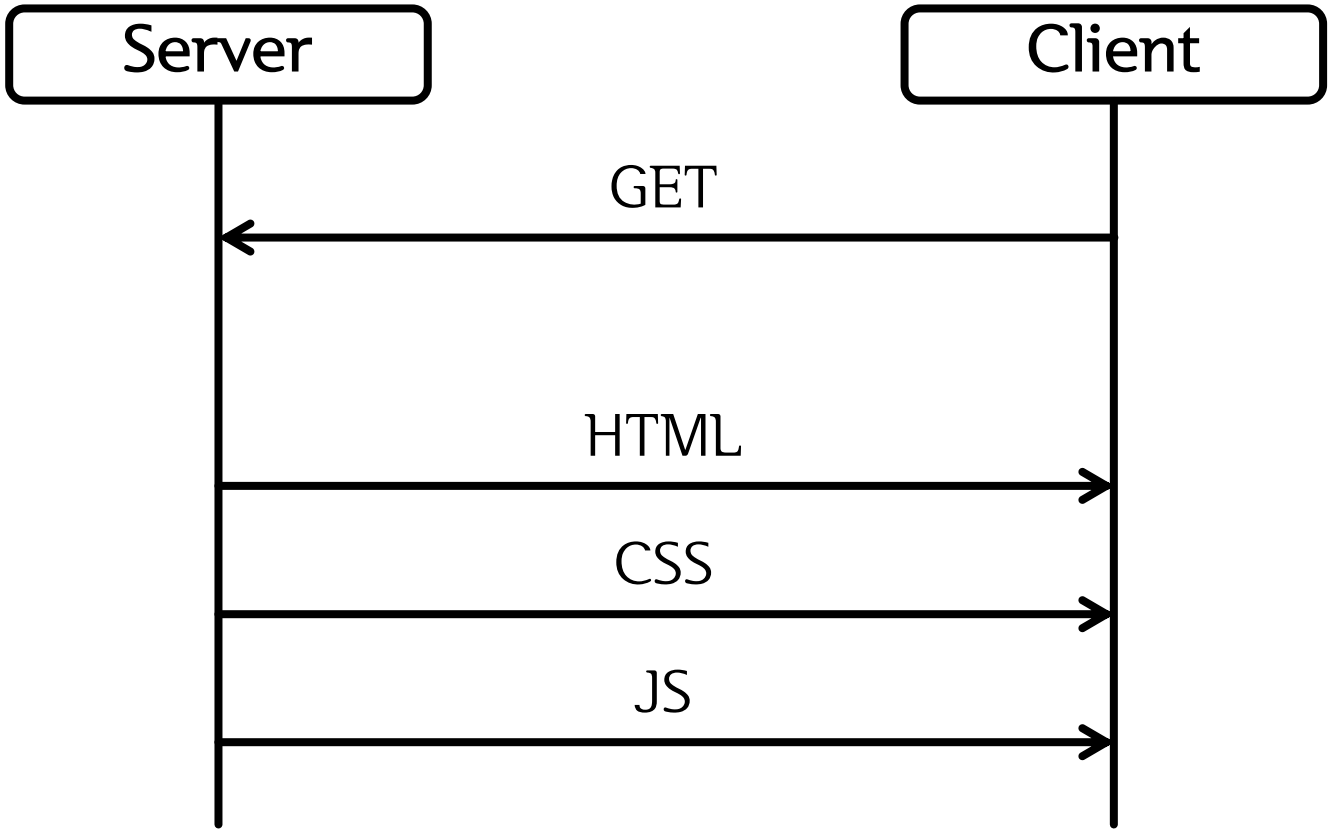
send(MSG_OOB)

```
➜  ✗ ./oob_recv 9000
Connected client 1
Urgent message: 0
exchange_data() : receive from 0.0.0.0 : 123456789
➜  ✗
```

```
➜  ✗ ./oob_send 127.0.0.1 9000
Connected...........
Input message(close or oob): oob
OK
ByeBye!!
➜  ✗
```

## readv(), writev() 함수 사용하기

- 시나리오



# readv(), writev() 함수 사용하기

Server

```c
int exchange_data(int clnt_sock)
{
    int str_len;
    char buf[BUF_SIZE] = {
        0,
    };
    char msg1[] = "HTML";
    char msg2[] = "CSS";
    char msg3[] = "JS";

    struct iovec vec[3];

    read(clnt_sock, &str_len, sizeof(int));
    str_len = read(clnt_sock, buf, str_len);
    buf[str_len] = 0;

    printf("Request from client: %s ", buf);

    vec[0].iov_base = msg1;
    vec[0].iov_len = strlen(msg1);

    vec[1].iov_base = msg2;
    vec[1].iov_len = strlen(msg2);

    vec[2].iov_base = msg3;
    vec[2].iov_len = strlen(msg3);

    str_len = writev(clnt_sock, vec, 3);
    puts("");
```

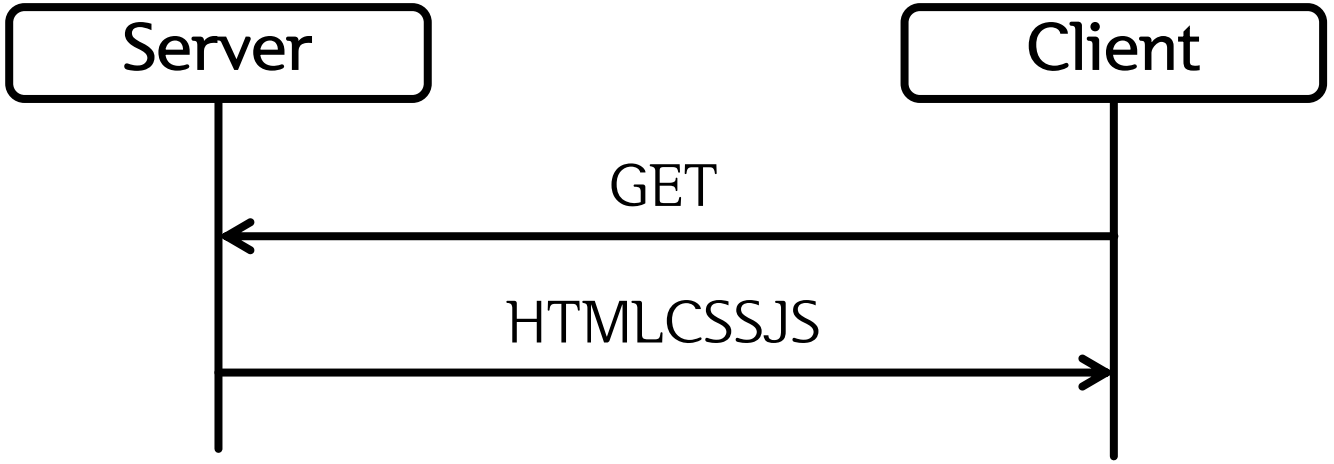
Client

```c
void exchange_data(int sock)
{
    int str_len;
    char req[] = "GET";
    char buf1[BUF_SIZE] = {
        0,
    };
    char buf2[BUF_SIZE] = {
        0,
    };
    char buf3[BUF_SIZE] = {
        0,
    };
    struct iovec vec[3];

    str_len = strlen(req);
    write(sock, &str_len, sizeof(int));
    write(sock, req, str_len);
    puts("Send Request GET");

    vec[0].iov_base = buf1;
    vec[0].iov_len = 4;

    vec[1].iov_base = buf2;
    vec[1].iov_len = 3;

    vec[2].iov_base = buf3;
    vec[2].iov_len = 2;

    readv(sock, vec, 3);

    printf("Read bytes: %d \n", str_len);
    printf("First message: %s \n", buf1);
    printf("Second message: %s \n", buf2);
    printf("Third message: %s \n", buf3);

    close(sock);
}
```

```
➜  ✗ ./writev_serv 9000
connected client: 7
Request from client: GET
Write bytes: 9
disconnected client: 7
```

```
➜  ✗ ./readv_clnt 127.0.0.1 9000
Connected..........
Send Request GET
Read bytes: 3
First message: HTML
Second message: CSS
Third message: JS
➜  ✗
```

## readv(), writev() 함수 사용하기




```
vec[0].iov_base = buf1;
vec[0].iov_len = sizeof(buf1);
```

```
➜                    ✗ ./writev_serv 9000
connected client: 7
Request from client: GET
Write bytes: 9
disconnected client: 7
```

```
➜                    ✗ ./readv_clnt 127.0.0.1 9000
Connected...........
Send Request GET
Read bytes: 3
First message: HTMLCSSJS
Second message:
Third message:
➜                    ✗
```

# 과제 #05

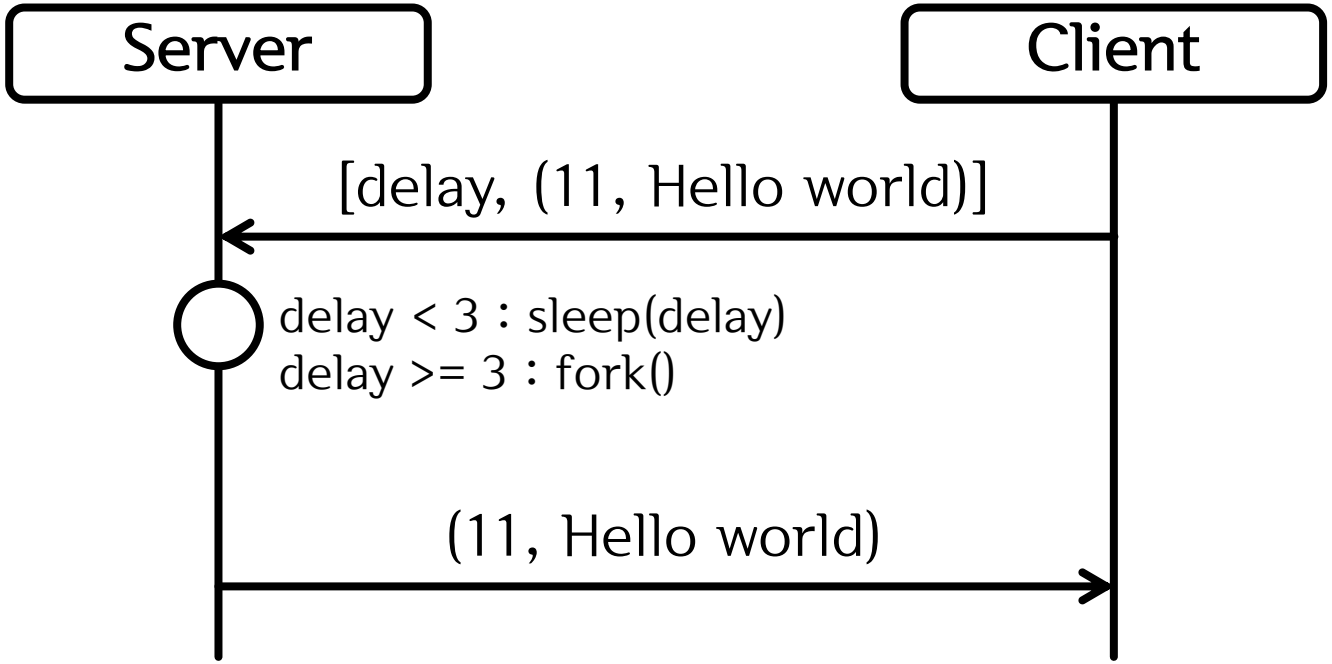
- **파일 내용을 전송하는 Server-Client 코드 작성**

  - **'hw05_server.c' server 프로그램 생성**
    - 멀티프로세스와 멀티플렉싱이 모두 지원되는 서버를 구현함.
      실행 예시) ./hw05_server 8080
    - Server는 다수의 Client에게 서비스를 제공할 수 있음
    - Client로 부터 메시지 수신 시 지연 시간을 함께 수신하며, 지연 시간 후 Client에게 받은 문자열을 되돌려줌

    (지연 시간이 2보다 크다면 자식 프로세스를 생성해서 해당 클라이언트 소켓과의 통신 및 서비스 제공 기능을 위임함)

  - **'hw05_client.c' client 프로그램 생성**
    - Client가 실행되면 콘솔을 통해 문자열과 지연시간을 입력 받음
    - 입력 받은 문자열과 지연시간을 서버에 전송하고 서버로부터 echo된 문자열을 출력함
    - 'q' 또는 'Q' 를 입력 받으면 프로그램을 종료함

# 과제 #05




- **주의 사항: 지연시간은 양수이며, 지연시간과 문자열을 전송하는 함수는 제공 예정**
- **제출 파일:** hw05_server.c, hw05_client.c, 실행 결과 스크린샷 이미지 (jpg, png 등)
- **제출 방법:** '학번_이름_HW05.zip'으로 압축하여 LMS 과제 게시판에 제출
- **제출 기한:** 2021년 1월 4일 (월) 13:59까지

- 실행 결과 스크린샷 이미지 예시

Server Program

```
➜                    ✗ ./hw5_server 8080
connected client: 7
connected client: 8
Created Child Process
receive from 127.0.0.1 : Hello world
receive from 127.0.0.1 on child: Hello world
```

Client Program

```
080
Connected...........
Enter a positive number for delay: 10
Input message(Q to quit): Hello world
Message from server: Hello world
Enter a positive number for delay:
```

Client Program

```
➜                    ✗ ./hw5_client 127.0.0.1
8080
Connected...........
Enter a positive number for delay: 2
Input message(Q to quit): Hello world
Message from server: Hello world
Enter a positive number for delay:
```